#Laboratorul 2

Simularea circuitelor

Qiskit ne permite sa simulam modul de functionare a calculatoarelor cuantice. Unitatea de bază a Qiskit este circuitul cuantic. Construire și Execuție:

- *Construire* - îți permite să creezi diferite circuite cuantice care reprezintă problema pe care o rezolvi;
- *Execuția* - îți permite să le rulezi pe diferite backend-uri. După ce joburile au fost executate, datele sunt colectate și procesate în funcție de rezultatul dorit si de problema pe care o ai de rezolvat.

+ Code + Text

```
!pip install qiskit
```

```
Collecting qiskit
  Downloading qiskit-1.4.2-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
Collecting rustworkx>=0.15.0 (from qiskit)
  Downloading rustworkx-0.16.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (10 kB)
Requirement already satisfied: numpy<3,>=1.17 in /usr/local/lib/python3.11/dist-packages (from qiskit) (2.0.2)
Requirement already satisfied: scipy>=1.5 in /usr/local/lib/python3.11/dist-packages (from qiskit) (1.14.1)
Requirement already satisfied: sympy>=1.3 in /usr/local/lib/python3.11/dist-packages (from qiskit) (1.13.1)
Collecting dill>=0.3 (from qiskit)
  Downloading dill-0.3.9-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: python-dateutil>=2.8.0 in /usr/local/lib/python3.11/dist-packages (from qiskit) (2.8.2)
Collecting stevedore>=3.0.0 (from qiskit)
  Downloading stevedore-5.4.1-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.11/dist-packages (from qiskit) (4.12.2)
Collecting symengine<0.14,>=0.11 (from qiskit)
  Downloading symengine-0.13.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.0->qiskit) (1.17.0)
Collecting pbr>=2.0.0 (from stevedore>=3.0.0->qiskit)
  Downloading pbr-6.1.1-py2.py3-none-any.whl.metadata (3.4 kB)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from sympy>=1.3->qiskit) (1.3.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from pbr>=2.0.0->stevedore>=3.0.0->qiskit) (75.1.0
Downloading qiskit-1.4.2-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (6.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.8/6.8 MB 21.4 MB/s eta 0:00:00
Downloading dill-0.3.9-py3-none-any.whl (119 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 119.4/119.4 kB 4.9 MB/s eta 0:00:00
Downloading rustworkx-0.16.0-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 24.5 MB/s eta 0:00:00
Downloading stevedore-5.4.1-py3-none-any.whl (49 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 49.5/49.5 kB 3.5 MB/s eta 0:00:00
Downloading symengine-0.13.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (49.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 49.7/49.7 MB 8.6 MB/s eta 0:00:00
Downloading pbr-6.1.1-py2.py3-none-any.whl (108 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 109.0/109.0 kB 2.8 MB/s eta 0:00:00
Installing collected packages: symengine, rustworkx, pbr, dill, stevedore, qiskit
Successfully installed dill-0.3.9 pbr-6.1.1 qiskit-1.4.2 rustworkx-0.16.0 stevedore-5.4.1 symengine-0.13.0
```

```
pip install qiskit_aer
```

```
import numpy as np
from qiskit import QuantumCircuit

#from qiskit import *
```

Constructia circuitului

Vom crea un circuit utilizand `QuantumCircuit`. Spre exemplu, daca dorim sa construim un circuit de doi qubiti, vom proceda astfel:

```
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit
c=ClassicalRegister(2,'cbit')
q=QuantumRegister(2,'qbit')
circ=QuantumCircuit(q,c)
'''# Create a Quantum Circuit acting on a quantum register of three qubits
circ = QuantumCircuit(2)'''

circ.x(0)
circ.s(1)
circ.cx(1,0)
```

```
circ.h(0)
circ.h(1)
print(circ)
```

```
     ┌───┐┌───┐┌───┐
qbit_0: ┤ X ├┤ X ├┤ H ├
     ├───┤└─┬─┘└───┘
qbit_1: ┤ S ├──■──┤ H ├
     └───┘     └───┘
cbit: 2/═══════════════
```

După ce creezi circuitul cu registrele sale, poți adăuga porți („operații”) pentru a manipula registrele.

Spre exemplu, sa construim un circuit ce produce starea lui Bell

$$|\beta_{00}\rangle = (|00\rangle + |11\rangle)/\sqrt{2}.$$

Pentru a crea aceasta stare, vom incepe cu un circuit de doi qubiti. To create such a state, we start with a three-qubit quantum register. Implicit, fiecare qubit este initializat cu starea $|0\rangle$. Deci inputul va fi $|00\rangle$. Pentru a crea starea lui Bell, vom aplica urmatoarele porti:

- Poarta Hadamard H pe qubitul $qbit_0$, ce ne pune in superpoziti2: $(|0\rangle + |1\rangle)/\sqrt{2}$.
- O poarta Controlled-NOT (C_X) cu controlul pe qubitul 0 si targetul pe qubit 1.

În Qiskit, operațiile pot fi adăugate în circuit una câte una, dupa cum este exemplificat mai jos:

```
# Adauga  H gate pe qubitul 0:
circ.h(0)
# Adauga poarta CX (CNOT) cu controlul pe  qubitul 0 si targetul pe qubitul 1:
circ.cx(0, 1)
```

```
<qiskit.circuit.instructionset.InstructionSet at 0x7c96989a5ab0>
```

Cum vizualizam circuitul?

Vom putea vizualiza circuitul folosind din Qiskit: `QuantumCircuit.draw()`, trasează circuitul în forma pe care o intalnim in cartile de specialitate. Sau mai simplu, pentru a vizualiza in terminal putem adauga `print(circ)`

```
pip install pylatexenc
```

```
Requirement already satisfied: pylatexenc in /usr/local/lib/python3.10/dist-packages (2.10)
```

```
from qiskit import *
import matplotlib.pyplot as plt


#circ.draw('mpl')
circ.draw(output="text")
#print(circ)
```

```
import matplotlib as mat
import pylatexenc
```

```
print(circ)
circ.draw('mpl')
```

Salvam circuitul

Putem salva imaginea obtinuta cu circuitul folosind '.savefig('nume fisier')'

```
circ.draw('mpl').savefig('figura.png')
```

```
#In caz de eroare la 'mpl' trebuie sa rulati si sa instalati:
pip install matplotlib
pip install pylatexenc
```

În acest circuit, qubiții sunt aranjați într-o ordine, cu qubitul zero în partea de sus și qubitul doi în partea de jos. Circuitul se citește de la stânga la dreapta (adică porțile aplicate mai devreme în circuit apar mai spre stânga).

IMPORTANT: Reprezentarea qubitilor in Qiskit

Presupunem ca avem un sistem de n qubiti, si qubitul i is reprezentat prin vectorul v_i. Qiskit foloseste o ordine în care qubitul n se află pe partea *stângă* a produsului tensorial, astfel încât vectorii de bază vor fi etichetați astfel: $v_{n-1} \otimes \cdots \otimes v_1 \otimes v_0$.

Spre exemplu, daca qubitul q_0 este in starea 0, si qubitul q_1 este in starea 1, Qiskit va reprezenta starea sistemului compus ca $|10\rangle$, invers fata de cum am reprezentat in scris ($|01\rangle$).

Aceasta scriere pe dos afecteaza si operatiile cu matrici.

Spre exemplu, reprezentarea in Qiskit a matricii corespunzatoare operatorului Controlled-X (C_X) avand qubitul de control qubitul 0 si targetul qubitul 2, va fi :

$$C_X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

Cum simulam circuite

Pentru a simula un circuit vom folosi modulul `quant_info`. Acest simulator, ne va afisa starea cuantica, adica un vector complex de dimensiune 2^n, unde n este numarul de qubiti pe care il are circuitul.

Pentru inceput, am putea sa vedem starea initiala a circuitului, sau daca dorim, sa o intializam cu o alta stare.

```
from qiskit.quantum_info import Statevector

# seteaza starea initiala un vector de intregi
state = Statevector.from_int(0, 2**2)
#dar daca vreau sa dau input un vector complex
import numpy
coef = [1j/numpy.sqrt(2), 0, 0, 1j/numpy.sqrt(2)]
state = Statevector(coef)

print(state)

''' Cum evolueaza acea stare dupa parcurgerea circuitului:
stare = state.evolve(circ)
print(stare)
print(circ)
#vizualizeaza starea in latex
#state.draw('latex')
'''
```

```
Statevector([0.+0.70710678j, 0.+0.j        , 0.+0.j        ,
             0.+0.70710678j],
            dims=(2, 2))
' Cum evolueaza acea stare dupa parcurgerea circuitului:\nstare = state.evolve(circ)\nprint(stare)\nprint(circ)\n#vizualizeaza starea i
n latex\n#state.draw('latex')\n'
```

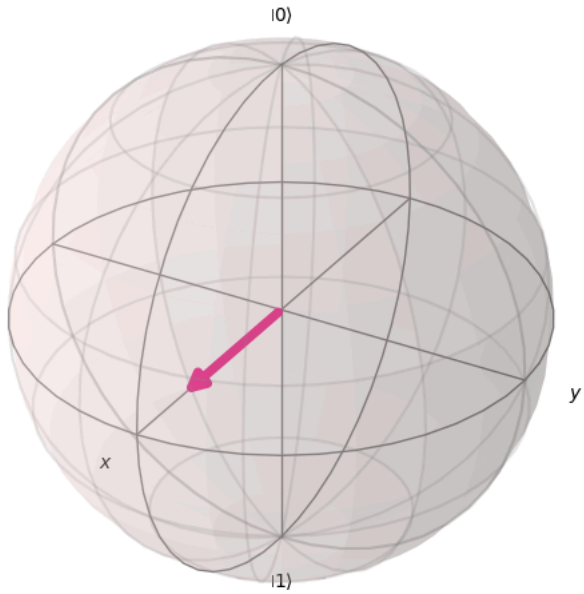
```
from qiskit.visualization import array_to_latex

#Sau alta alternativa de a reprezenta in Latex starea
array_to_latex(state)
```

$$\begin{bmatrix} \frac{\sqrt{2}i}{2} & 0 & 0 & \frac{\sqrt{2}i}{2} \end{bmatrix}$$

Qiskit ne permite sa vizualizam pe sfera lui Bloch starile, utilizand `plot_bloch_vector`.

```
from qiskit.visualization import plot_bloch_vector
from qiskit.quantum_info import Statevector
stare=[1/np.sqrt(2),0,0,-1/np.sqrt(2)]
plot_bloch_vector(stare)
```



Reprezentarea matricilor in circuit

Qiskit, prin modulul `quant_info` ne permite sa vedem matricea unitara asociata circuitului implementat. Daca spre exemplu, avem un circuit de n qubiti, matricea asociata circuitului va fi de $2^n \times 2^n$.

```
from qiskit.quantum_info import Operator

U = Operator(circ)

# Afiseaza matricea
U.data

print(U)
# Get the matrix representation of the circuit
matrix_rep = U.to_matrix()

# Print the matrix
print("Matrix representation of the circuit:")
print(matrix_rep)
```

```
Operator([[ 0.5+0.j ,  0.5+0.j ,  0. +0.5j,  0. +0.5j],
          [-0.5+0.j ,  0.5+0.j ,  0. +0.5j,  0. -0.5j],
          [ 0.5+0.j ,  0.5+0.j ,  0. -0.5j,  0. -0.5j],
          [-0.5+0.j ,  0.5+0.j ,  0. -0.5j,  0. +0.5j]],
         input_dims=(2, 2), output_dims=(2, 2))
Matrix representation of the circuit:
[[ 0.5+0.j   0.5+0.j   0. +0.5j  0. +0.5j]
 [-0.5+0.j   0.5+0.j   0. +0.5j  0. -0.5j]
 [ 0.5+0.j   0.5+0.j   0. -0.5j  0. -0.5j]
 [-0.5+0.j   0.5+0.j   0. -0.5j  0. +0.5j]]
```

Rularea circuitelor pe simulatoare

Simulatoarele folosite anterior sunt utile deoarece oferă informații despre starea generată de circuit (in mod ideal) și despre reprezentarea matriceală a acestuia.

Totuși, un experiment real se încheie prin măsurarea fiecărui qubit (de obicei în baza computațională $|0\rangle, |1\rangle$ basis). Daca nu am avea masuratoare nu am putea afla informatie despre stare. Am vazut in cadrul cursului ca masuratoarea colapseaza starea.

Presupumen ca facem masuratori asupra ambilor qubiti in starea lui Bell $|\beta_{00}\rangle$

$$|\beta_{00}\rangle = (|00\rangle + |11\rangle)/\sqrt{2},$$

si fie xy sirul de biti rezultat in urma masuratorii. Amintim ca, din cauza etichetarilor din qiskit, bitul x va corspunde qubitului 1, si bitul y qubitului 0.

Probabilitatea sa obtinem bitii xy se calculeaza:

$$\mathrm{P}(xy) = |\langle xy|\beta_{00}\rangle|^2$$

deci probabilitatea sa obtinem 00 sau 11 va fi 1/2.

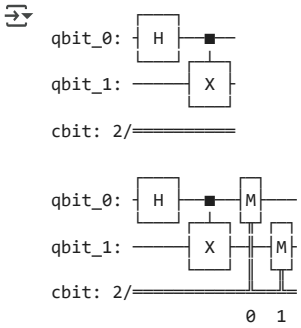
Pentru a simula un circuit care include măsurători, trebuie să adăugăm măsurători în circuit, si și să folosim un backend diferit din Aer.

```
#cream un alt circuit
print(circ)
#Masurare
circ.measure(0,0)
circ.measure(1,1)
print(circ)



'''
circ1 = QuantumCircuit(2, 2)
#delimitam circuitul
circ1.barrier(range(2))
# masuram si trecem rezultatele in biti clasici
circ1.measure(range(2), range(2))

# In Qiskit putem compune circuite
# Pentru a face asta, circ1 ar trebui sa fie primul si front=True
qc = circ1.compose(circ, range(2), front=True)
'''
#Desenarea circuitului
#qc.draw('mpl')
```




```
'\ncirc1 = QuantumCircuit(2, 2)\n#delimitam circuitul\ncirc1.barrier(range(2))\n# masuram si trecem rezultatele in biti clasici\ncirc1.
measure(range(2), range(2))\n\n# In Qiskit putem compune circuite\n# Pentru a face asta, circ1 ar trebui sa fie primul si front=True\nq
```

Acest circuit adaugă un registru clasic și doua măsurători care sunt folosite pentru a evidentia rezultatul qubiților în biții clasici.

Pentru a simula acest circuit, vom folosi `qasm_simulator` din Qiskit Aer. Fiecare rulare a circuitului ne va da fie sirul de biti 00 sau 11. Pentru a obtine informatii legate de distributiile de probabilitate a sirurilor de biti, ar trebui sa rulam circuitul de un numar mai mare de ori.

Putem repeta rularea circuitului de un numar de ori folosind `shots`.

```
# Adaugarea transpiler-ului
#În Qiskit, un quantum transpiler este folosit pentru a transforma un circuit cuantic
# într-un set de operații care pot fi executate pe un anumit backend (hardware cuantic).
from qiskit import transpile

# Simulatorul AerSimulator
from qiskit_aer import AerSimulator

backend = AerSimulator()

qc_comp= transpile(circ, backend)

# Executia circuitului pe  qasm simulator.
```

```
# Vom rula circuitul de  1024 de ori, care este valoarea implicita, de fapt.
job_sim = backend.run(qc_comp, shots=10000)

#Luam rezultatele din job
result_sim = job_sim.result()
```

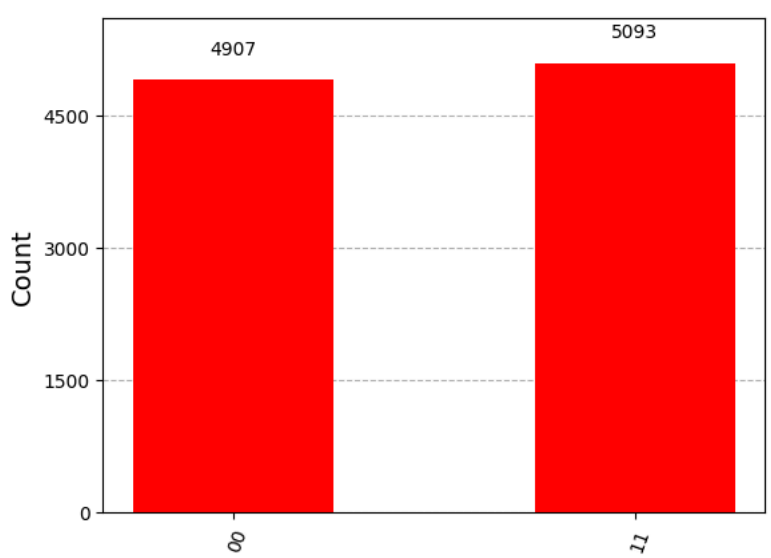
Odată ce avem rezultatele, vom putea vedea de cate ori am obtinui o anumita stare utilizand `get_counts(circuit)`.

```
counts = result_sim.get_counts(qc_comp)
print(counts)
```

```
{'00': 4907, '11': 5093}
```

Deci, aproximativ 50% outputul este 00 si 50% este 11. Qiskit ne permite sa vedem rezultatele si sub forma de histograme, folosind functia `plot_histogram`.

```
from qiskit.visualization import plot_histogram
plot_histogram(counts, color="red")
```



Exercitii propuse Laboratorul 2

Exercitiul 1.

- Construiti un circuit care actioneaza pe un registru de 3 qubiti.
- Măsurați fiecare qubit. Care este rezultatul?
- Aplicați poarta X asupra qubitului 0. Analizați rezultatul.
- Afișați starea obținuta.

```
Start coding or generate with AI.
```

Exercitiul 2.

Implementati urmatoarele circuite:

Circuit 1

https://drive.google.com/file/d/15xV-DMekVl4bCNihoLGCiCLrX-3P7u2Y/view?usp=drive_link

Circuit 2

https://drive.google.com/file/d/1UD3YxuE-jmk7GMNSiGyb0hdPJWPGZXVG/view?usp=drive_link

1. Pentru fiecare dintre circuitele de mai sus, afisati matricea asociata circuitului, vectorul de stare obtinut dupa parcurgerea circuitului.
2. Masurati ambii qubiti si reprezentati histograma cu rezultatele obtinute.
3. Dar ati incerca sa masurati doar primul qubit al circuitelor, ce rezultate obtineti?

```
Start coding or generate with AI.
```

Exercitiul 3.

Implementati in Qiskit strile lui Bell $|\beta_{10} = \frac{1}{\sqrt{2}}|00\rangle - |11\rangle$ si $|\beta_{01}\rangle = \frac{1}{\sqrt{2}}|01\rangle + |10\rangle$.

Ce obtineti in urma masurarii in baza computationala, dar in urma masurarii in baza Bell ?(masurarea in Baza Bell se face cu ajutorul circuitului corespunzator)

```
Start coding or generate with AI.
```

Exercitiul 4

Implementati algoritmul lui Deutsch-Jozsa pentru $n = 3$.

```
Start coding or generate with AI.
```